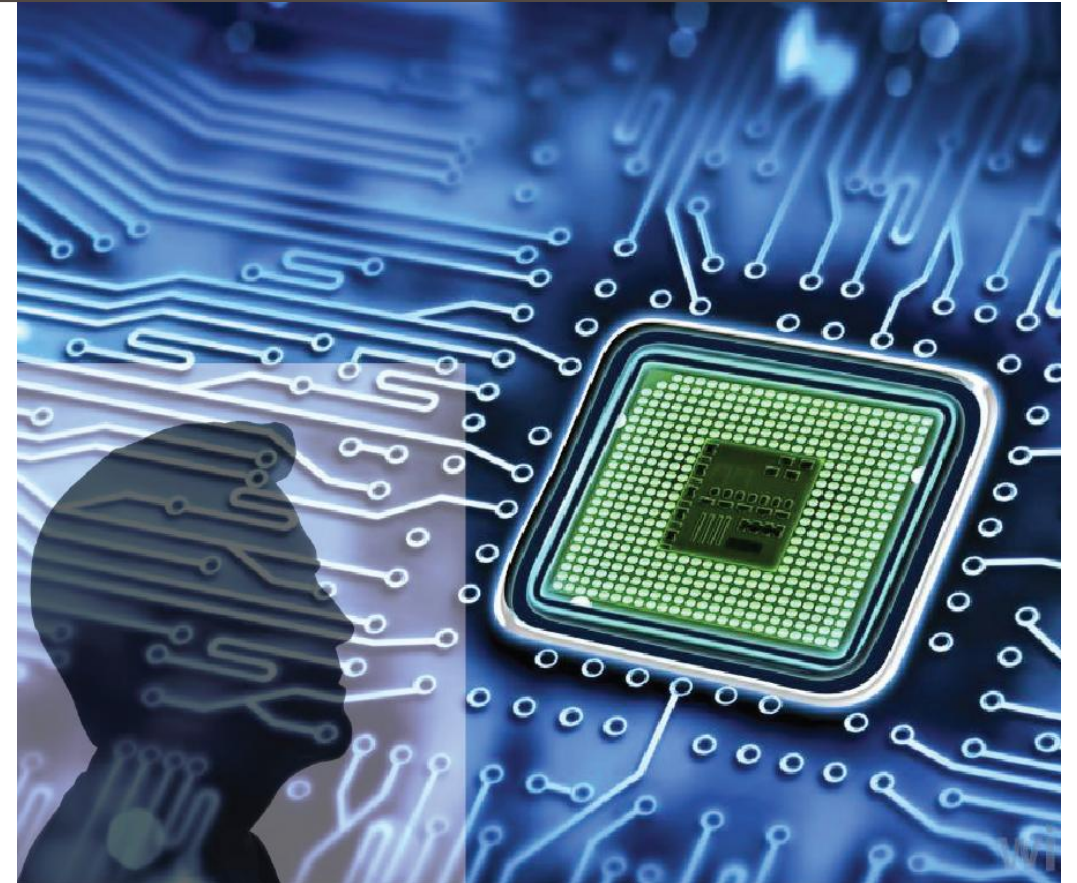


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-12 Lecture-2 SystemVerilog Assertions

Sneh Saurabh
13th November 2025



Sequences: Non-consecutive Repetition Operator

Non-consecutive repetition operator: $\text{Expr}[=N]$

- Some expression Expr occurs N times, need not be consecutive
- The match ends at the $N\text{th}$ occurrence
- There can be additional cycles after the $N\text{th}$ occurrence before the sequence continues

Example:

- $\text{sig_1}[=2] \## 1 \text{ complete}$
- sig_1 should repeat two times with any number of clock cycles in between, and then *complete* should be high any number of clock cycles later

Sequences: Goto Repetition Operator

Non-consecutive repetition operator: `Expr[->N]`

- Some expression `Expr` occurs `N` times, need not be consecutive
- The match ends at the `Nth` occurrence
- The sequence should continue immediately after the `Nth` occurrence (goes to the next part immediately).

Example:

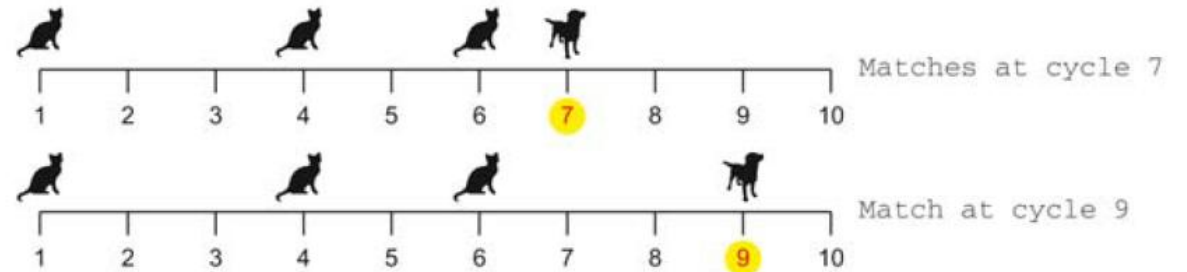
- `sig_1[->2] ##1 complete`
- `sig_1` should repeat two times with any number of clock cycles in between, and then `complete` should be high just in the next cycle

Sequences: Non-consecutive Repetition and Goto Repetition Operator

```
cat [=3] ##1 dog
```

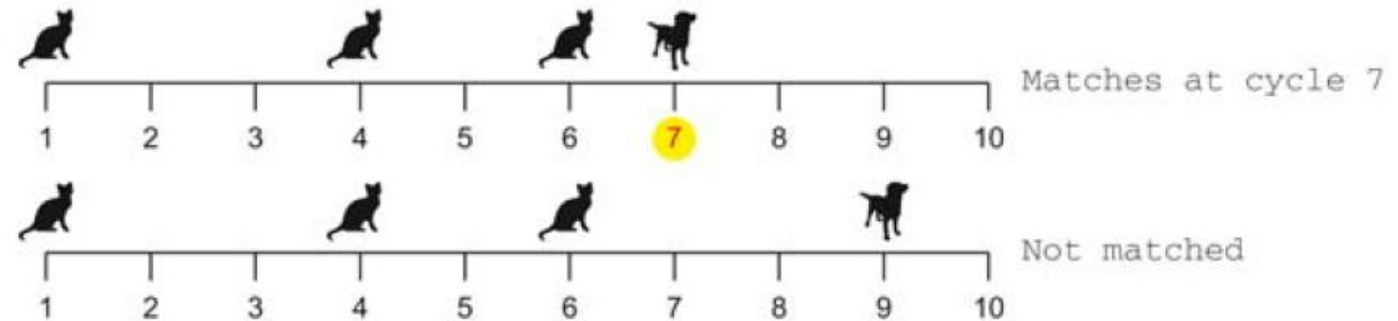
3 cats (can be non-consecutive)
followed by dog sometime after
the last cat

cat[=3] ##1 dog : A sequence with 3 cats, and a dog sometime after



```
cat [->3] ##1 dog
```

3 cats (can be non-consecutive)
followed by dog after the last cat



Sequences: Non-consecutive Repetition and Goto Repetition Operator

- Non-consecutive repetition operator $\text{Expr}[=N]$ and Goto Repetition $\text{Expr}[->N]$ operators can take only Boolean expressions and cannot take sequences
- It is in contrast to the consecutive repetition operator $A[*N]$

If a is a Boolean expression

- $a[/math> $\neq$ $5]$ is valid$
- $a[->5]$ is valid
- $a[*5]$ is valid

If a and b are Boolean expressions

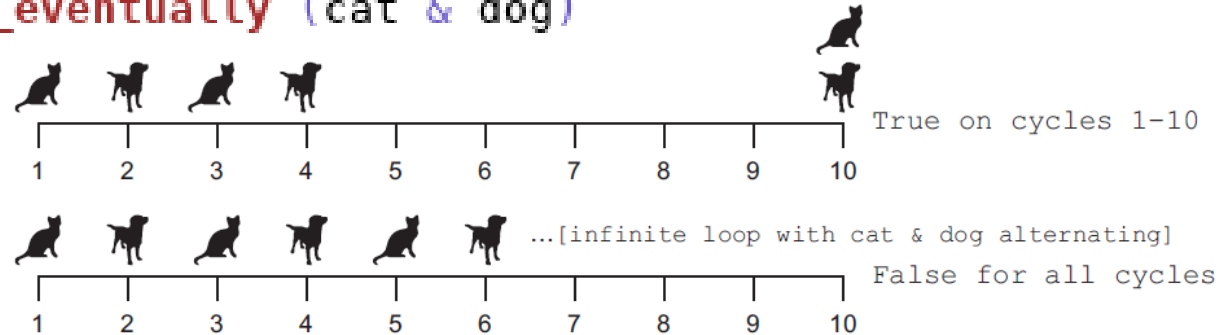
- $(a \ \#\# \ 1 \ b)[\neq 5]$ is invalid
- $(a \ \#\# \ 1 \ b)[->5]$ is invalid
- $(a \ \#\# \ 1 \ b)[*5]$ is valid

- This restriction exists because the semantics of non-consecutive matching become very complex when applied to multi-cycle sequences

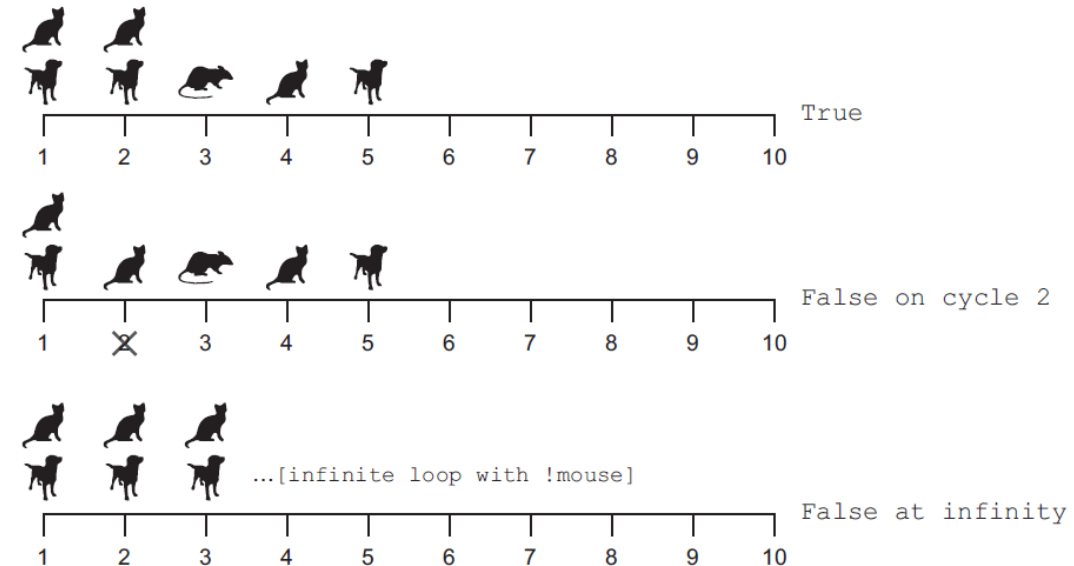
Concurrent Assertions: LTL operators

- Linear Temporal Logic (LTL) operators are needed to model liveness properties
- s_eventually**
 - **s_eventually** S
 - S must occur at some time
- s_until**
 - A **s_until** B
 - A must be true until B (which must occur) is true

s_eventually (cat & dog)



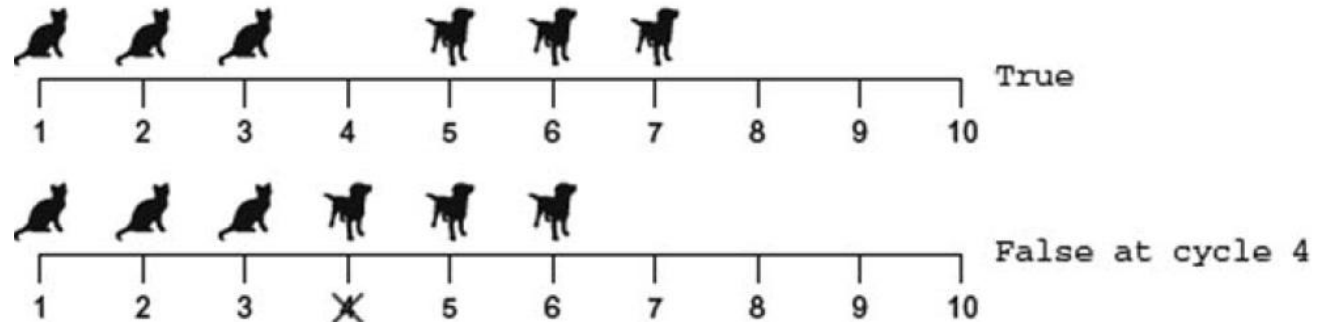
(cat **| ->** dog) **s_until** mouse



Negated Sequences as Property

- Negated sequences: **not** operator preceding a sequence
- Negated sequence is a property, rather than a sequence
 - It cannot be used as an antecedent (LHS) of an implication operator
- Useful to check if a particular sequence never occurs
 - Define safety properties

`!(cat ##1 dog)`



Named Sequences and Properties

- SystemVerilog allows us to write **named** sequences and properties and use them subsequently
- Using keywords: **sequence**, **endsequence**, **property**, **endproperty**

```
sequence within_5_cycles(sig);  
    ##[1:5] sig;  
endsequence
```

```
property b_within_5_after_a(a, b);  
    a |-> within_5_cycles(b);  
endproperty
```

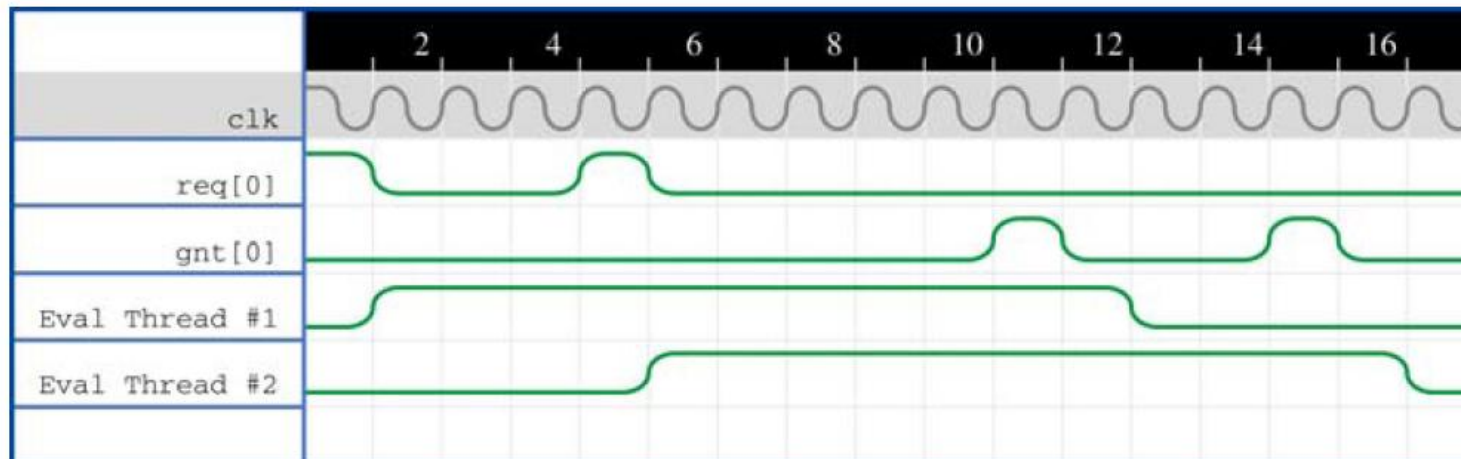
```
named_prop: assert property  
    (b_within_5_after_a(grant[0], !req[0]));
```

```
unnamed_prop: assert property  
    (grant[0] |-> ##[1:5] !req[0]);
```

Assertions: Implicit Multithreading

- If multiple behaviors overlap in the assertion, SVA has built-in (implicit) multithreading
 - A new thread is implicitly created whenever the starting condition is met
 - Multiple behaviors will be evaluated simultaneously

```
delayed_gnt:  
  assert property  
    (req[0] |-> ##10 gnt[0]);
```



- `req[0]` is high in 1st cycle and 5th cycle
- Two evaluation threads are launched processed concurrently (these threads do not interfere)

Connecting Properties with Design

Two ways of connecting properties with a design (RTL):

- Directly inserting in the RTL
 - As demonstrated in earlier examples
- Using **bind** statement
 - Allows separation of RTL from properties
 - Properties added in a standalone module (separate from RTL)
 - Using **bind** statement instance of a property module is created inside the design (RTL)

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
  always @(posedge clk or negedge reset)
  begin
    if (reset)
      grant <= 2'b00;
    else if (req[0])
      grant <= 2'b10;
    else if (req[1])
      grant <= 2'b10;
    else
      grant <= 2'b00;
  end
endmodule
```

```
module arbiter_fpv(input bit clk, input logic [1:0] grant);
  check_grant: assert property(@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simulataneously!");
endmodule

bind arbiter arbiter_fpv arbiter_fpv_inst(clk, grant);
```

Universal Verification Methodology (UVM)

UVM: Basics

- Universal Verification Methodology (UVM) is a standardized framework for functional verification using SystemVerilog
- Popular in industry for complex designs



<https://www.accellera.org/community/uvm>

Features

- **Reusability:** components reusable across projects, teams, and companies due to standardization
- **Scalability:** deployable on large designs
- **Automation:** stimulus generation, transaction handling, and checking.

UVM Architecture: Testbench

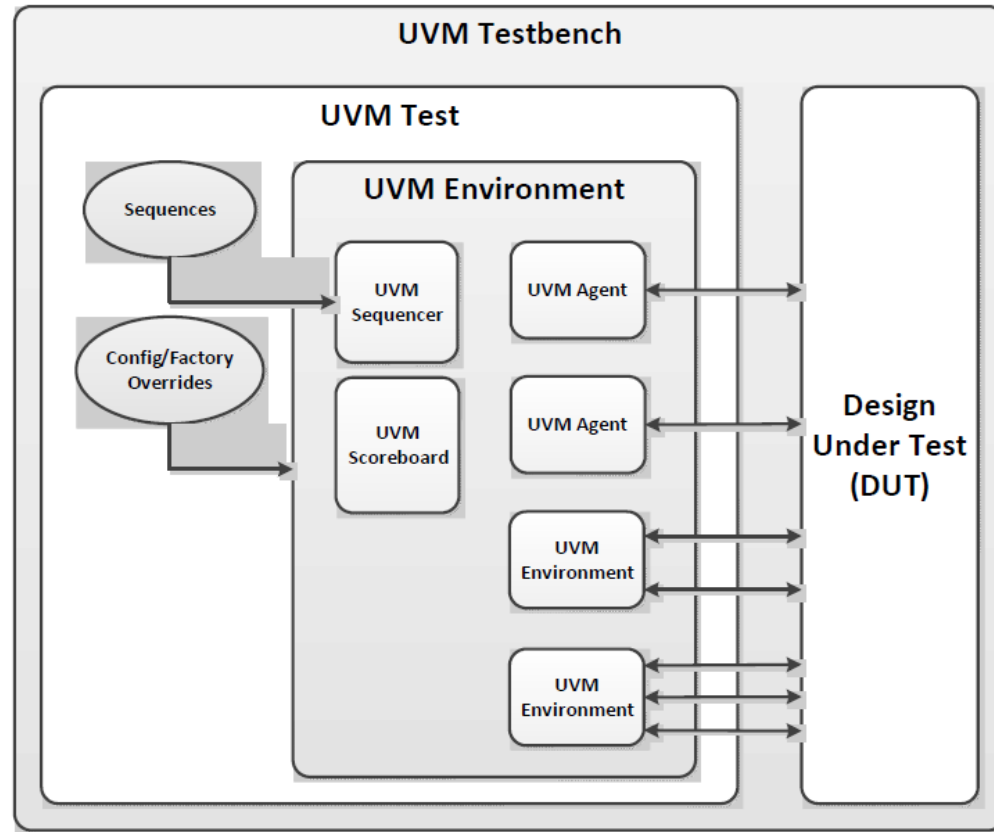


Figure 1—Typical UVM Testbench Architecture

UVM Testbench (top-level entity)

- Instantiates DUT and UVM Test class
 - Configures their connections
- UVM Test is dynamically instantiated at run-time
 - Allows the UVM Testbench to be compiled once and run with many different tests

UVM Architecture: Test (**uvm_test** class)

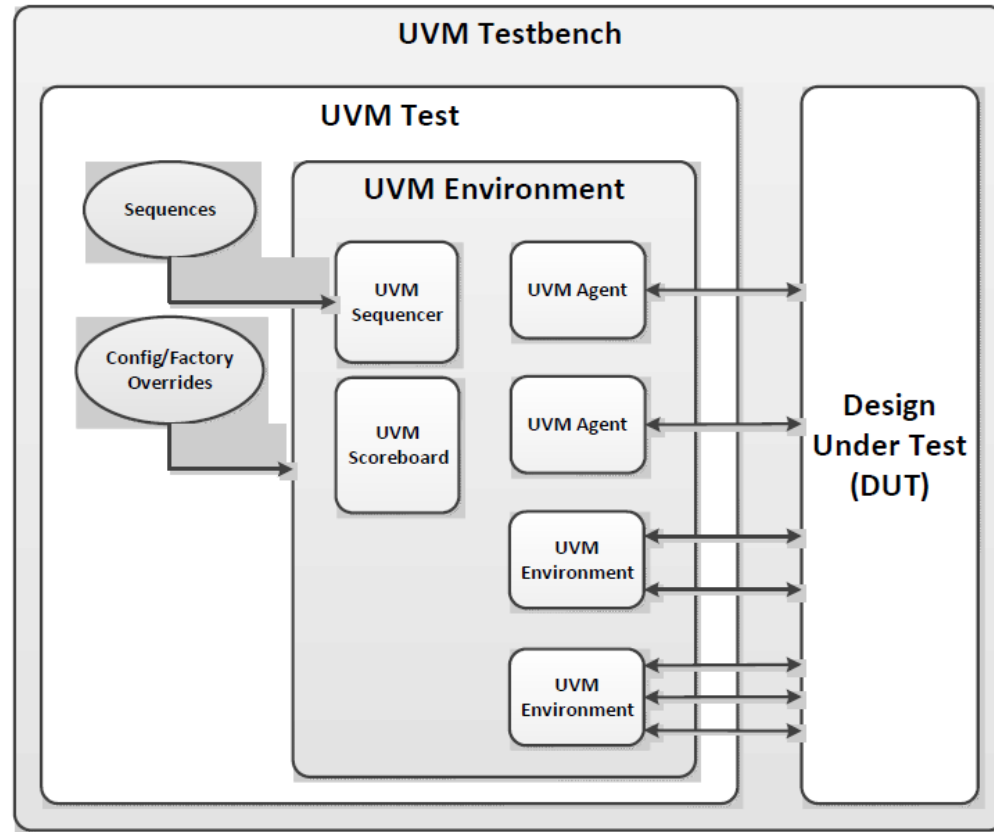


Figure 1—Typical UVM Testbench Architecture

UVM Test

- Three main functions:
 - Instantiates the top-level environment
 - Configures the environment (via factory overrides or the configuration database)
 - Applies stimulus by invoking UVM sequences through the environment
- Typically, we create one base class for UVM Test with the UVM Environment instantiation
 - Other tests will extend this base test, configure the environment differently or select different sequences to run
 - Testbench can instantiate different tests

References

- Seligman, Erik, Tom Schubert, and MV Achutha Kiran Kumar. Formal verification: an essential toolkit for modern VLSI design. Elsevier, 2023
- Saurabh, Sneha. Introduction to VLSI design flow. Cambridge University Press, 2023.

- **Universal Verification Methodology (UVM) 1.2 User's Guide**

https://www.accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf